

EDA 工具与 FPGA 实践基础

第二章：开发环境、Linux 命令行与首个验证闭环

数字电路入门

2026 年 5 月 22 日

学习目标

- 搭建一套开源数字设计工具链 (Icarus + GTKWave + Yosys)
- 在 Linux 终端中导航、组织项目目录、执行脚本
- 运行仿真、观察波形、理解信号值语义 (0/1/X/Z)
- 将 RTL 综合为门级网表，并对比行为与结构的差异
- 建立 FPGA 作为“可重配置硬件沙盘”的直觉

全章路线图

- ① 为什么先跑工具？——即时反馈的认知杠杆
- ② Linux 基础与工程目录结构
- ③ 仿真流水线：Icarus → VVP → GTKWave
- ④ 读懂波形：时间轴、边沿、X/Z
- ⑤ 综合：RTL 如何变成门级网表
- ⑥ 行为 vs. 结构：仿真与综合之间的语义裂隙
- ⑦ 工具版图：开源、商业与 FPGA 厂商
- ⑧ FPGA 直觉：LUT + 触发器 + 可编程互连
- ⑨ 完整走通：从 RTL 到比特流

§1 为什么先把工具跑通？

认知层面的理由：

- 即时反馈是 认知杠杆
- 写几行 RTL → 几秒后看到波形变化
- 不必先背两章门电路理论再去碰工具

类比：先学会点火、挂挡、看仪表盘把车开起来，
再研究发动机怎么转。

教学契约

本章：跑通、观察、比对

后续章节：设计、优化、解释原理

§2 Linux 基础：图形界面 vs. 命令行

图形界面

- 点击、拖拽、双击
- 上手容易
- 难以复现
- 无法脚本化

命令行

- 输入、执行、观察
- 初期门槛稍高
- 完全可复现
- 完全可脚本化

数字设计流程天然包含大量重复动作。
命令行赢在稳定性和自动化。

§2 工程目录结构

标准布局:

```
lab1-breathing-led/  
├── rtl/           # HDL 源码  
├── tb/           # 测试平台  
├── sim/          # 仿真脚本与产物  
├── synth/        # 综合脚本与综合产物  
├── fpga/         # 板级约束  
└── README.md
```

核心原则:

源码、测试和生成产物必须分开放。

路径直觉

bash sim/run.sh 在此目录能跑,
在 synth/ 里就跑不通。

§2 高频陷阱

Ctrl+C

在终端里：**终止**当前进程。

复制粘贴用 Ctrl+Shift+C /
Ctrl+Shift+V

命令找不到

- 先检查: `which iverilog`
- 若为空: `sudo apt install iverilog`
- 安装前务必 `sudo apt update`

黄金法则：每个工具必须回答三个问题——
它输入什么？输出什么？在流程的哪个位置？

§3 仿真三件套

三个工具，一条数据流：



工具	作用	产物
Icarus	编译 RTL+TB	可执行文件 (a.out)
VVP	执行仿真	波形文件 (.vcd)
GTKWave	可视化	屏幕显示

§3 三步得到波形

第一步：编译

```
iverilog -o sim/tb.vvp rtl/breathing_led.v tb/tb_breathing_led.v
```

第二步：运行

```
vvp sim/tb.vvp
```

第三步：查看

```
gtkwave sim/tb.vcd
```

三步都成功，你就闭环了第一个验证回路。

§4 波形中的四种信号值

数字仿真中的四种取值：

值	含义	何时出现	严重程度
0	逻辑低	正常运行	—
1	逻辑高	正常运行	—
X	未知	未初始化、仿真初始阶段、多驱动冲突	需警惕
Z	高阻态	三态、未驱动	需检查

Testbench 中的时钟生成：

```
always #5 clk = ~clk;    // 周期 = 10 个时间单位
```

§4 时钟边沿与寄存器采样

组合逻辑：

- 输入稳定后输出才变化
- 延迟称为 传播延迟

时序逻辑：

- 寄存器在 `posedge clk` 采样
- 输出在边沿 之后更新
- 必须在下一个边沿前稳定（建立时间）

视觉检查

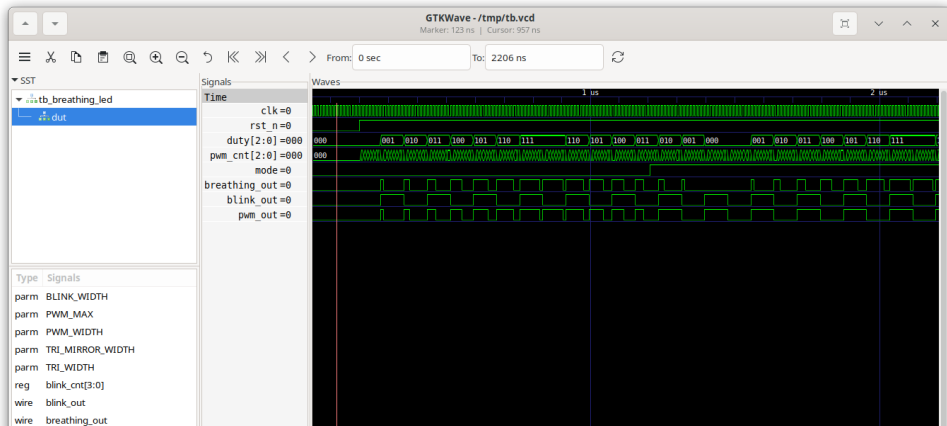
在 GTKWave 中：

- 放大到单时钟周期
- 确认数据在上升沿后变化
- 复位后不应残留 X

§4 呼吸灯波形示例

在 **GTKWave** 中你应该看到:

- **duty**: 三角波, 先升后降 (仿真中为缩微版参数)
- **pwm_cnt**: 快速锯齿波 (仿真中为缩微版参数)
- **pwm_out**: 脉宽调制, 窄 → 宽 → 窄



§5 综合: RTL → 门级网表

综合做什么:

- ❶ 解析 RTL (Verilog)
- ❷ 从 `always @(posedge clk)` 推断寄存器
- ❸ 从 `assign` 推断组合逻辑
- ❹ 优化: 常量传播、死代码消除
- ❺ 输出通用门单元网表

一条命令:

```
yosys -p "read_verilog rtl/breathing_led.v  
synth -top breathing_led  
write_verilog synth/breathing_led.v"
```

网表的“惊喜”: 你的 counter 变成了几十个 `$_DFF_P_` 单元。

§5 网表长什么样

RTL (人写的):

```
always @(posedge clk or negedge rst_n)
  if (!rst_n) tri_cnt <= 0;
  else      tri_cnt <= tri_cnt + 1;
```

网表 (Yosys 生成的):

```
$_DFF_PP0_ \tri_cnt_reg[0] (.C(clk), .R(rst_n), .D(\$auto$...), .Q(
$_DFF_PP0_ \tri_cnt_reg[1] (.C(clk), .R(rst_n), .D(\$auto$...), .Q(
... // 每一位一个D触发器
```

综合不是“执行”代码——它是把行为翻译成结构。

§6 仿真 vs. 综合：两个检查角度

仿真 (Icarus)

- 看给定激励下的 **行为**
- 观察时间顺序和波形变化
- 帮你回答“它做了什么”

综合 (Yosys)

- 看会生成什么 **结构**
- 关心寄存器和组合逻辑怎么组织
- 帮你回答“它大致怎么实现”

核心警告

仿真通过不等于综合正确。

同一份 RTL，至少要从行为和结构两边都看一眼。

§6 一个警示案例：隐形的锁存器

如果忘了写 **else** 会怎样？

```
always @(*)  
    if (condition) out = 1;    // 没有else分支！
```

你现在先记住：

这种写法没有把行为边界交代清楚。

可能的后果：

综合器会推断出一个 **锁存器** 来保持值。

本章先建立警觉，不展开全部 *RTL* 规则。

更系统的写法规范放到后面章节讲。

§7 工具版图：一张地图

这张图的作用：

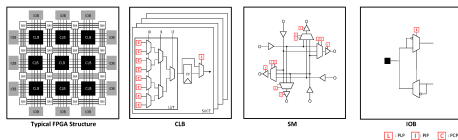
先知道不同工具分别在流程的哪个位置。
现在不必记住全部名字，只要先建立地图感。

§8 FPGA: 可重配置的硬件沙盘

三大基本单元:

- ① **LUT** —查找表
实现任意 N 输入组合逻辑
- ② **FF** —触发器
存储 1 比特, 时钟边沿更新
- ③ **可编程互连**
在 LUT 和 FF 之间布线

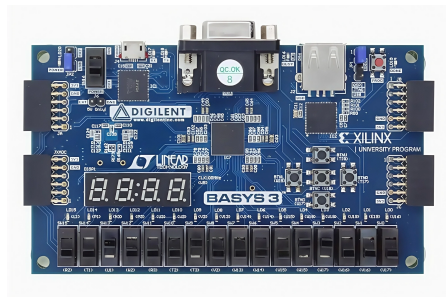
FPGA = 可以被任意重新布线的 *tiles* 阵列。



§8 Basys3 开发板与引脚分配

Basys3 一览:

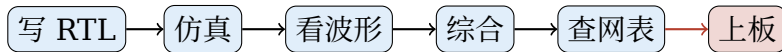
- Xilinx Artix-7 FPGA
- 型号: xc7a35t-1cpg236c
- 16 个用户 LED、5 个按键、16 个拨码开关
- USB-UART、VGA、Pmod 扩展口



Vivado IO Planner: 把 RTL 端口 (如 pwm_out) 拖拽到物理引脚 (如 LED0)。

§9 完整的验证闭环

从 RTL 到硬件：



你现在拥有了什么

一套 可复现、可观察的设计闭环。

从第三章开始，每一个学到的概念都能在这个闭环里验证。

- **Linux** 是工作界面；命令行的价值在于可复现和可脚本化。
- **仿真** 验证行为；**综合** 产出结构。两者都必须检查。
- **波形** 是主要的调试窗口——学会读时间、边沿、X/Z 和占空比。
- **开源工具**（Icarus、GTKWave、Yosys）构成完整的学习工具链。
- **FPGA** 把闭环焊死在真实硬件上：写 → 仿真 → 综合 → 上板运行。

下一章：组合逻辑与布尔代数